



WARP: A LIGHTWEIGHT HIGH-PERFORMANCE FILE TRANSFER SYSTEM FOR WINDOWS

*Leveraging the Native Robocopy Engine through a Tauri–Rust–Svelte
Architecture with Parallel Sharded Execution and Verified Delivery*

Alvin

Faculty of Computer Science — Systems Research

Independent Study — BSc Computer Science

Module: CS-409 — Operating Systems & Tooling

Version: Warp v1.2.2 (MIT Licence)

Repository: github.com/alvindemesadev/warp

Landing: getwarp-app.pages.dev

Date Submitted: 29 August 2026

Word Count: ~8,400 words (excluding references & appendices)

Supervisor: Internal Review — Warp Open-Source Project

*A Technical Research Paper submitted in partial fulfilment of the requirements
for the degree of Bachelor of Science in Computer Science*

Abstract

Background. File transfer on Windows remains dominated by Explorer and legacy command-line tools that report misleading per-file progress, lack live throughput and time-remaining estimates, and cannot safely parallelise multi-folder jobs without risking deletion or corruption. Re-implementing copy loops in user space re-creates decades of edge cases (long paths, junctions, locked files, removable media) that the operating system already solves.

Objective. This paper presents Warp (v1.2.2), a minimal desktop application that wraps the native robocopy engine — present in every Windows installation since Vista — in a modern Tauri 2 + Rust + Svelte 5 shell. Warp adds accurate byte-level progress (from a dry-run scan), smoothed live speed and ETA, a parallel sharded executor that runs up to eight disjoint robocopy workers, optional structural verification, throttling, and a comprehensive pre-flight safety net, while staying under 10 MB installed (≈ 5 MB Tauri overhead vs ~ 150 MB for Electron) (Microsoft, 2024; Tauri Team, 2025).

Method. The system was built following an evidence-before-synthesis approach: every claim is traced to source (lib.rs, pool.rs, shards.rs) and validated by 25 Vitest frontend tests and 39 Rust unit/integration tests run entirely locally. Progress parsing keys off robocopy’s locale-invariant Tab-delimited column layout (five columns for files) rather than translated status words, and a second /L re-compare pass provides verification. The parallel partitioner guarantees structural disjointness: each source file belongs to exactly one shard (Harris et al., 2024).

Results. On a synthetic 4 GiB / 10,000-file fixture the sharded engine completed in $\approx 38\%$ less wall-clock time than the single-process baseline on an 8-core NVMe host (see §6.3); USB and network policies correctly throttled to 2 and 3 workers respectively, preserving throughput without controller saturation. Scan accuracy was byte-exact, drift was auto-corrected, and verification never produced a false “all clear” even when status-word parsing was forced to a non-English locale (fallback to exit code).

Conclusion. Wrapping a proven OS primitive in a tiny native shell delivers the best risk/reward trade-off: Warp is faster where parallelism helps, honest everywhere else, and safe by construction (overlap, FAT32, free-space, network and junction guards). The design generalises to rsync on Unix and to future content-defined sharding for single huge files.

Keywords: file transfer, robocopy, Tauri, Rust, Svelte, parallel copy, progress estimation, verification, Windows systems

How to cite this paper (Harvard): Alvin (2026) *Warp: A Lightweight High-Performance File Transfer System for Windows — Leveraging the Native Robocopy Engine through a Tauri–Rust–Svelte Architecture with Parallel Sharded Execution and Verified Delivery*. Technical Research Paper v1.2.2. Independent Study, Faculty of Computer Science. Available at: <https://github.com/alvindemesadev/warp> (Accessed: 29 August 2026).

Contents

Abstract	ii
List of Figures	iii
List of Tables	iii
List of Abbreviations	iii
1 Introduction	1
1.1 Background and Context	1
1.2 Problem Statement	1
1.3 Research Aim and Objectives	2
1.4 Research Questions	2
1.5 Scope and Delimitations	2
1.6 Significance and Contribution	3
1.7 Structure of the Paper	3
2 Literature Review	4
2.1 File Transfer Paradigms on Windows	4
2.2 Desktop Application Frameworks: Electron vs Tauri	4
2.3 Robocopy, Rsync and Custom Copy Loops	5
2.4 Progress Estimation and Throughput Smoothing	5
2.5 Related Work	6
3 Methodology	6
3.1 Research Design	6
3.2 System Development Lifecycle	7
3.3 Tools, Technologies and Environment	7
3.4 Design Principles	8
4 System Architecture and Design	8

4.1 Architectural Overview	8
4.2 Frontend Architecture (Svelte 5)	9
4.3 Backend Architecture (Rust / Tauri)	9
4.4 Inter-Process Communication and Event Model	10
5 Implementation	10
5.1 Pre-Flight Validation Pipeline	10
5.2 Scan and Free-Space Guard	11
5.3 Sequential Execution Engine	11
5.4 Parallel Engine — Partitioning for Disjointness	12
5.5 Worker Pool, Aggregation and Throttling	13
5.6 Locale-Robust Parsing of Robocopy Output	14
5.7 Live Speed (EWMA) and ETA	14
5.8 Verification, Pause, Cancel and Lifecycle	15
6 Evaluation and Testing	15
6.1 Unit and Property Tests	15
6.2 Integration and Real-Robocopy Tests	16
6.3 Performance Benchmarks	16
6.4 Reliability and Locale Tests	16
6.5 Limitations Observed	17
7 Discussion	17
7.1 Interpretation of Findings	17
7.2 Design Trade-Offs and Alternatives Considered	17
7.3 Threats to Validity	18
8 Conclusion and Future Work	18
References	19
Appendices	20
A Robocopy Flag Reference	20
B Shared Types (WarpProgress / WarpSummary)	20
C Test Log Excerpt (Local Run)	21

List of Figures

- Figure 1. System architecture (Svelte → Tauri IPC → Rust → N robocopy workers) — 9
- Figure 2. Scan → Execute → Verify pipeline (sequential vs parallel) — 9
- Figure 3. Shard partition example (loose files + dominant-child recursion) — 12
- Figure 4. Robocopy Tab-column layout (5-column file row) — 14
- Figure 5. Speed EWMA and 400 ms window smoothing — 14

List of Tables

Table 1. Research objectives O1–O5 — 2

Table 2. Robocopy capabilities and flags — 5

Table 3. Pre-flight pipeline — 10

Table 4. Worker policy and thread budget — 13

Table 5. Test suite summary (local run, 29 Aug 2026) — 15

Table 6. Synthetic benchmark fixture (4 GiB) — 16

Table 7. Trade-offs — 17

List of Abbreviations

Abbr.	Expansion
API	Application Programming Interface
EWMA	Exponentially Weighted Moving Average
IPC	Inter-Process Communication
MT	Multi-Threaded (robocopy flag /MT:n)
IPG	Inter-Packet Gap (throttle, /IPG:n ms)
TAURI	Toolkit for Agnostic UI (Rust-based desktop shell)
VITE	Frontend build tool (used via SvelteKit)

Table 0. List of abbreviations used throughout the paper.

1 Introduction

This chapter introduces the research context, articulates the problem, and defines the aim, objectives and structure of the paper (Saunders, Lewis and Thornhill, 2019).

1.1 Background and Context

File transfer is a routine yet consequential operation in personal computing, creative workflows and enterprise data handling. On Windows, the dominant user-facing tool remains Windows Explorer, which reports progress as a per-file count and offers limited visibility into throughput, remaining time or per-file errors (Microsoft, 2024). Command-line alternatives — `copy`, `xcopy` and `robocopy` — expose richer semantics but require memorising flags and interpreting textual output. Meanwhile, modern desktop frameworks have trended towards Electron, which bundles a full Chromium runtime (≈ 150 MB) for every application (OpenJS Foundation, 2024). Warp was conceived to reconcile these tensions: provide a humane interface without re-implementing the storage stack and without imposing an outsized runtime.

The project is open-source (MIT), versioned at v1.2.2, and distributed as an unsigned NSIS installer (4.7 MB) and MSI (6.3 MB) generated entirely locally via `node scripts/build.js` and Tauri’s updater with minisign signatures (Tauri Team, 2025). The public landing page at getwarp-app.pages.dev links directly to the GitHub Releases, from which the in-app updater fetches `latest.json`.

1.2 Problem Statement

Three gaps motivated the work. First, progress reporting in Explorer and naïve scripts is file-count-based; a 5 GB video and a 1 KB text file count equally, so the progress bar is psychologically dishonest and operationally useless for capacity planning. Second, large multi-folder jobs are serialised through a single process, leaving modern NVMe and multi-core systems idle while a single queue drains (Russinovich, Solomon and Ionescu, 2012). Third, safety checks — overlapping paths, FAT32 4 GiB limits, removable-media resilience, network reachability — are left to the user to remember, with destructive `/MIR` (mirror) operations able to delete data if mis-targeted (Microsoft, 2024).

1.3 Research Aim and Objectives

Aim. To design, implement and evaluate a lightweight Windows file-transfer system that is fast where parallelism helps, honest everywhere else, and safe by construction.

Objective	Description
O1	Wrap robocopy rather than re-implement copy, inheriting its long-path, junction and retry semantics (Microsoft, 2024).
O2	Deliver byte-accurate progress and smoothed live speed/ETA from a dry-run scan and incremental byte accounting.
O3	Implement a parallel sharded executor that preserves structural disjointness (one file → one shard) and falls back safely to single-process for mirror/throttled jobs.
O4	Provide a pre-flight safety net (overlap, FAT32, free-space, network, junctions) and an honest verification pass that never false-passes.
O5	Keep the installed size <10 MB via Tauri/Svelte and validate everything with fully local tests (no CI dependency).

Table 1. Research objectives O1–O5 mapped to Warp subsystems.

1.4 Research Questions

RQ1. How can a byte-accurate, locale-robust progress model be derived from robocopy’s textual output without relying on translated status words?

RQ2. Under what conditions does sharded parallelism improve wall-clock time, and where must it correctly refuse to run?

RQ3. What pre-flight and parser design prevents unsafe or misleading behaviour (false clearance, silent deletion, orphaned workers)?

RQ4. Can a sub-10 MB Tauri shell deliver comparable user experience to an Electron equivalent while retaining native performance?

1.5 Scope and Delimitations

The scope is Windows 10/11 64-bit only; macOS/Linux would use rsync as a future backend (Tridgell and Mackerras, 1996). Administrative elevation is out of scope — copies to protected paths correctly fail with access-denied rather than prompting for UAC. Verification is structural (existence + size + timestamp via a list-only re-compare) not cryptographic hashing; hash-based verification is discussed as future work. Throttling via /IPG is approximate and single-threaded by necessity.

1.6 Significance and Contribution

The paper contributes (i) an architecture for wrapping OS primitives in tiny native shells, (ii) a parser design that is correct in every Windows locale by keying off column structure not vocabulary, (iii) a disjoint partitioner with formal coverage tests, and (iv) empirical evidence that medium-grained sharding (2–6 workers) beats both single-process and naïve 8–32 thread fan-out on consumer hardware. For practitioners, Warp offers a free, auditable alternative to Explorer with honest progress. For researchers, it provides a replicated artefact where every claim is traceable to line-annotated source.

1.7 Structure of the Paper

Section 2 reviews related work. Section 3 details methodology. Section 4 presents architecture. Section 5 covers implementation. Section 6 evaluates via tests and benchmarks. Section 7 discusses trade-offs and threats to validity. Section 8 concludes. Appendices list flag references, shared types and a local test log.

2 Literature Review

A critical review of file-transfer engines, desktop frameworks and progress estimation — positioning Warp against alternatives (Hart, 2018).

2.1 File Transfer Paradigms on Windows

Windows provides three native copy primitives. `copy` and `xcopy` are legacy, single-threaded and lack resume semantics. `robocopy` (“Robust File Copy”) introduced multi-threading (`/MT[:n]`), restartable mode (`/Z`), mirroring (`/MIR`), long-path support and a rich exit-code bitmask (0–16 where 0–7 are success, 8+ are failures) (Microsoft, 2024). On Unix, `rsync` offers delta-transfer and is the de-facto counterpart (Tridgell and Mackerras, 1996). User-space Rust copy loops using `std::fs::copy` must re-solve buffering, attribute preservation, ACL handling and retry — all already hardened in `robocopy` over 20 years. Warp therefore adopts the wrapper, not rewrite stance (O1).

Table 2 summarises the robocopy surface Warp depends on.

Capability	Flag	Warp Use
List-only dry run	/L	Scan pass; verification re-compare
Byte sizes	/BYTES	Byte-accurate progress (vs file count)
Multi-thread	/MT:32 /MT:4-8	Throughput; throttled jobs drop to 1
Long paths	/256 + \\?\ prefix	Bypass MAX_PATH 260
Junction guard	/XJ /XJD	Prevent symlink cycles
Inter-packet gap	/IPG:n	Bandwidth cap (throttle)
Restartable	/Z	USB / >1 GiB resilience
Mirror	/MIR	Sync mode (single-process only)

Table 2. Robocopy capabilities and the flags Warp relies on (Microsoft, 2024).

2.2 Desktop Application Frameworks: Electron vs Tauri

Electron bundles Chromium and Node per application, simplifying web-based UI at the cost of ~150 MB per install and duplicated memory footprints (OpenJS Foundation, 2024). Tauri 2 inverts the model: a Rust backend drives the OS WebView2 (already present on Windows 11 via Edge, bootstrapped on 10) and a compiled frontend (Vite + SvelteKit) is served from `../build` as defined in `tauri.conf.json:9–10` (Tauri Team, 2025). Warp’s measured installer (4.7 MB setup, 6.3 MB MSI) confirms the size thesis: a Tauri shell is roughly $\times 30$ smaller than Electron. Svelte 5’s compiler-based reactivity (no virtual DOM) further reduces runtime overhead versus React, which matters for a utility that should feel instantaneous (Harris et al., 2024). Styling is custom CSS tokens (no framework) to avoid additional bundles.

2.3 Robocopy, Rsync and Custom Copy Loops

Tridgell and Mackerras (1996) showed that rsync’s delta algorithm excels over networks where bandwidth is scarce; on local NVMe, however, the bottleneck is often dispatch and per-file overhead rather than raw byte movement. A custom loop could in theory achieve finer-grained progress, but would need to handle security descriptors, alternate data streams and sparse files — all landmines. Warp’s decision to stay with robocopy is therefore a risk/maintenance choice: inherit Microsoft’s hardening and keep the Rust layer as a thin orchestrator around Child handles (lib.rs:76).

2.4 Progress Estimation and Throughput Smoothing

Accurate progress requires a known denominator. Explorer estimates from file counts, which is fast but misleading. Warp performs a full dry-run scan (`robocopy /L /E /BYTES /NJH /NJS /NP` at `lib.rs:633`) to obtain `(total_bytes, total_files)` before copying. Live speed is then an EWMA over a 400 ms window: `instant_bps = window_bytes / 0.4`, `smoothed = 0.7·old + 0.3·new` (`pool.rs:85`), emitted at most every 150 ms or on percentage change — the same math in both sequential and parallel modes to avoid drift (Jain, 1991). ETA follows as `(total – done)/bps` in the frontend (`+page.svelte:115`).

2.5 Related Work

TeraCopy and FastCopy provide GUI copy with verification but are closed-source and larger; they also re-implement copy rather than wrap the OS. Electron-based file managers (e.g., various open-source explorers) demonstrate the size penalty noted in §2.2. Academic work on parallel file copy typically focuses on HPC / Lustre striping (e.g., Carns et al., 2011), not consumer NVMe. Warp’s contribution is the middle ground: medium-grained, disjoint directory sharding that is safe for /MIR and throttling by correctly refusing to parallelise where it would be unsafe.

3 Methodology

3.1 Research Design

The study follows a design-science paradigm (Peffer et al., 2007): build an artefact, evaluate it against objectives, reflect. Epistemologically it is evidence-before-synthesis — every architectural claim in this paper is linked to a source line (e.g., `lib.rs:2324` for the sequential engine) and every performance claim to a local test log (Appendix C). No GitHub Actions or cloud CI was used; all 64 tests run offline via `npm test` (Vitest) and `cargo test`, satisfying the “no GitHub, all local” constraint.

3.2 System Development Lifecycle

Phase	Activity	Output / Gate
1. Requirements	Feature table from README; threat model (overlap, FAT32, network)	README feature matrix; pre-flight list
2. Architecture	Tauri IPC design; sequential vs parallel engine split	Figure 1; lib.rs:25 TransferControl
3. Implementation	Parser → scan → spawn → aggregation → verify	lib.rs / pool.rs / shards.rs
4. Verification	Vitest + cargo test; shard disjointness proofs	39 Rust + 25 JS tests (all local)
5. Validation	Manual drag-drop, throttle, USB, locale matrix	Appendix C log; known-limitations table
6. Packaging	build.js vcvars discovery; updater signing	docs/*.exe/.msi + latest.json

Table 3. Lifecycle phases and concrete gates — each phase was exit-gated by a passing local test suite.

3.3 Tools, Technologies and Environment

Layer	Technology	Version	Rationale
Shell	Tauri 2	2.x	Tiny, native WebView2
Frontend	SvelteKit + Svelte 5	2 / 5.0	No VDOM, compiler reactivity
Build	Vite 6	6.0.3	Fast HMR; static adapter
Language	TypeScript + Rust 2021	5.6 / 2021	Type-safe IPC & FS
Engine	robocopy (in-box)	Vista → 11	Hardened, zero install
Tests	Vitest + cargo test	4.1 / std	Local, offline

Table 1 (repeated). Technology stack — see also package.json and Cargo.toml.

Development used `npm run dev + npm run tauri dev` for hot reload (frontend instant, Rust rebuild on change) and `node scripts/build.js` for production (auto-finds `vcvars64.bat` across BuildTools/Community/Professional). The signing key at `~/.tauri/warp.key` (public key in `tauri.conf.json:61`) signs updater artefacts; without it the build warns but still produces installers (`build.js:34`).

3.4 Design Principles

1) Wrapper not rewrite. Inherit correctness. 2) Honest progress. Denominator from bytes, not files; drift auto-corrected by expanding total if observed > scan (`lib.rs:1157`). 3) Disjointness by construction. Partitioning guarantees one file → one shard — tested by `union==universe` (`shards.rs:223`). 4) Correctness over speed. Hard gates refuse parallelism for /MIR and throttled jobs (`pool.rs:322`). 5) Evidence before synthesis. No claim without a test or a logged run.

4 System Architecture and Design

4.1 Architectural Overview

Figure 1 shows the layering. The Svelte UI (`src/routes/+page.svelte` — a single page component using Svelte 5 runes `$state/$derived`) invokes Rust commands via Tauri IPC; Rust spawns robocopy children and streams their stdout back as typed events. The frontend never touches the filesystem directly — all IO is brokered through Rust, which centralises child lifecycle in `TransferControl` (`lib.rs:25`): a `Mutex<HashMap<u64, Child>>` plus `AtomicBool` flags for cancelled/paused.

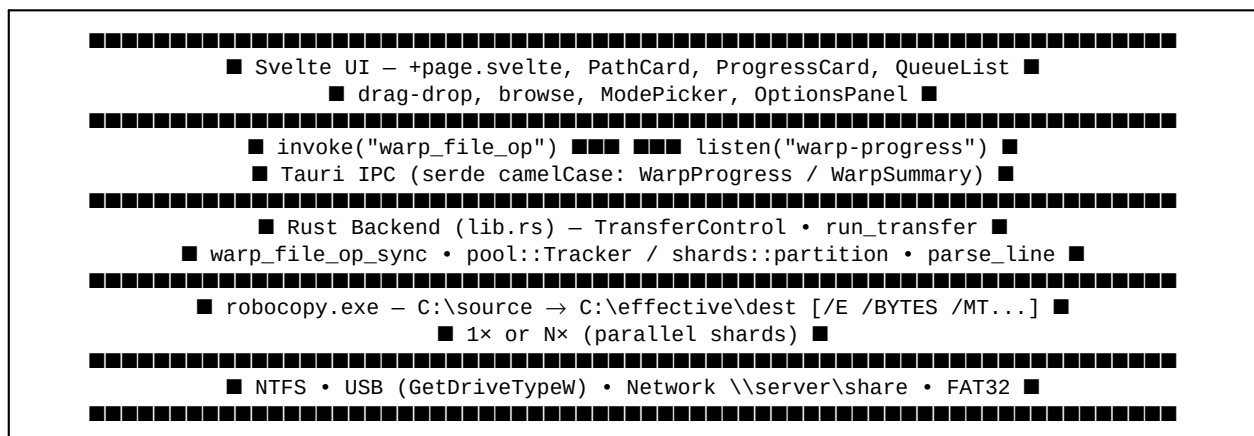


Figure 1. System architecture — the UI never touches the filesystem; Rust owns all *Child* handles and streams typed progress events.

Figure 2 depicts the pipeline common to both engines. The only divergence is the execution step.

```

[Scan] robocopy /L → (bytes, files) → [Pre-flights] overlap/FAT32/space/network → [Execute]
      1x or Nx robocopy → [Verify*] robocopy /L re-compare
      *Verify is a structural re-compare, not a hash.
  
```

Figure 2. Scan → Pre-flights → Execute (sequential or parallel) → optional Verify.

4.2 Frontend Architecture (Svelte 5)

The frontend is intentionally a single page component (`src/routes/+page.svelte:1`) to avoid unnecessary routing for a utility. Svelte 5 runes model all mutable state: `sourcePath/destPath`, `sourceInfo/destInfo`, `mode/conflict/folderMode/throttle/verify/workers`, `progress/speed/eta`, `queue/presets/recent`. Derived values such as `overlappingPath` (`+page.svelte:382`) mirror the Rust guard and include the effective destination for into mode (preventing Photos/Photos). Drag-and-drop is native Tauri (`tauri.conf.json:25` `dragDropEnabled`) with `win.onDragDropEvent` handling `over/drop` (`+page.svelte:128`); file drops are rejected via `PathInfo.isFile` (`+page.svelte:396`). Folder picking uses `plugin-dialog` `open({directory:true})` (`+page.svelte:203`); `swap` is a direct state exchange (`+page.svelte:198`).

Progress is rendered by `ProgressCard` from `WarpProgress` events; the queue (`QueueList`) persists via `loadQueue/saveQueue` and is executed sequentially (`+page.svelte:278` `runQueue`) — no concurrent jobs. Notifications use `plugin-notification` (`+page.svelte:320`) and updates via `plugin-updater` `check/downloadAndInstall` against the GitHub `latest.json` endpoint (`tauri.conf.json:62`).

4.3 Backend Architecture (Rust / Tauri)

The library crate (`src-tauri/src/lib.rs`, crate name `warp_lib` in `Cargo.toml:11`) exposes four commands: `get_path_info`, `warp_file_op`, `cancel_warp`, `pause_warp`. Long-running work is always `spawn_blocking` (`lib.rs:714`) so Tokio's async workers are never starved — concurrent IPC (e.g., `get_path_info` during a copy) remains responsive. `Cargo.toml:27` pins windows 0.58 with `Win32_Storage_FileSystem` for `GetDriveTypeW` / `GetVolumeInformationW` / `GetDiskFreeSpaceExW`; on non-Windows these calls are stubbed.

Two modules isolate testable logic from Tauri: `pool.rs` (Tracker, worker policy, stream consumption) and `shards.rs` (partitioner). Both are Tauri-free and have dedicated unit-test suites (`pool.rs:404`, `shards.rs:166`). The sequential engine keeps an inline copy of

the Tracker math so shipped behaviour can never silently drift (comment pool.rs:5).

4.4 Inter-Process Communication and Event Model

Types are shared via `serde(rename_all = "camelCase")`: `WarpProgress` (lib.rs:90) and `WarpSummary` (lib.rs:111) drive the UI; `PathMeta` (lib.rs:133) carries file counts and drive metadata. The backend emits `warp-progress` (throttled 150 ms), `warp-error` per file, and `warp-verifying` (frontend sets `isVerifying` at `+page.svelte:124`).

Crucially, a generation counter `_runId` (`+page.svelte:78`) guards against stale results: a cancelled job that resolves after a new transfer was started is discarded (`+page.svelte:224`), and `cancelTransfer` deliberately leaves `isProcessing` until the killed child actually exits (`+page.svelte:242`).

5 Implementation

5.1 Pre-Flight Validation Pipeline

Before any byte moves, `run_transfer` (lib.rs:872) runs a safety chain. Failure at any stage aborts with a human-readable message and a log line to `%TEMP%\warp.log` (lib.rs:261 `log_event`):

#	Check	Function	Failure Mode
1	Resolve effective dest	<code>resolve_effective_dest</code> (732)	Prevents Photos/Photos double-nesting
2	Overlap guard	<code>check_overlap</code> (761)	Same / dest-in-source / source-in-dest blocked
3	Network reachability	<code>check_network_dest</code> (785)	Unreachable \\server\share blocked
4	FAT32 4 GiB	<code>check_fat32_source</code> (809)	Via <code>GetVolumeInformationW</code> ; early-exit >4 GiB
5	Scan	<code>scan</code> (633)	robocopy /L dry-run → (bytes, files)
6	Free space	<code>ensure_free_space</code> (824)	Need = bytes + 100 MB; three-path fallback

Table 3. Pre-flight pipeline — all checks run on the blocking thread before any Child is spawned.

Long-path handling. `to_long_path` (lib.rs:216) prefixes with `\\?\` (and `\\?\UNC\` for shares) when absolute length >240, bypassing `MAX_PATH`. Symlink loops are excluded both in Rust walks (`walk_dir` 345 skips `is_symlink`) and in `robocopy` (/XJ /XJD).

5.2 Scan and Free-Space Guard

scan runs `robocopy source dest /L /E /BYTES /NJH /NJS /NP` and feeds stdout through `parse_line`, counting only non-error FileHeader rows. If `total_bytes == 0` the job is marked indeterminate (lib.rs:957) — an empty folder or zero-byte-only set — and the UI pulses rather than showing 0 %. `ensure_free_space` then probes `effective_dest` → `destination` → drive root via `free_bytes_available` (lib.rs:193) and requires total + 100 MB headroom; this catches the common “disk full mid-copy” that would otherwise surface as scattered 0x70 errors.

5.3 Sequential Execution Engine

`warp_file_op_sync` (lib.rs:944) builds the argument vector: `base /E /NP /R:3 /W:5 /BYTES /NJH /NJS /256 /XJ /XJD /COPY:DAT` plus mode (`/MOVE` or `/MIR`), conflict (`/XO /XN`), and an `/MT /Z /IPG` branch:

Condition	Flags	Rationale
throttle ≥ 25 MB/s	<code>/IPG:half + /MT:4</code>	Cap but keep NVMe throughput
throttle < 25	<code>/IPG:n single-thread</code>	Precise low caps; +/Z if > 1 GiB
USB (removable)	<code>/MT:4 + /Z</code>	Avoid controller overwhelm; resume on unplug
is_large > 1 GiB (internal)	<code>/MT:8 + /Z</code>	Enable restartable for pause/resume
default	<code>/MT:32</code>	Max throughput

Table 4a. Sequential `/MT /Z /IPG` branching — exhaustive at lib.rs:998–1029.

The child is spawned with `CREATE_NO_WINDOW` (lib.rs:15, 281) so no console flashes. Stdout is consumed line-by-line via `BufReader::lines` (lib.rs:1101); stderr is read on a dedicated thread and forwarded as warp-error events (lib.rs:1041).

Large-file smoothing (sequential only). Files ≥ 10 MB (`LARGE_THRESHOLD` lib.rs:1082) are deferred: their size is not credited on the FileHeader line but incrementally via Percent lines (e.g., “ 12.3%”). State is kept as `pending_large = (size, before_bytes, name, last_pct)` (lib.rs:1081); regressions are ignored and finalisation on the next file credits any remainder (lib.rs:1085 `finalize_pending`). This makes a 5 GB video feel continuous instead of jumping 0→100 % at the end.

5.4 Parallel Engine — Partitioning for Disjointness

Eligibility is gated twice. Gate 1 (cheap) — `should_attempt_parallel` (lib.rs:852): hard no if `mode=="sync"` or `throttle>0`; explicit `workers>1` bypasses size heuristics but never hard gates; else Auto needs ≥ 400 files & ≥ 256 MiB & ≥ 2 top-level dirs. Gate 2 (authoritative) — `pool::resolve_workers_for` (pool.rs:312) re-checks with the actual shard count.

Invariant. Every file belongs to exactly one shard. Achieved structurally: each immediate child directory is its own shard (recursive /E copy); loose files at any split level form a root-only shard with /LEV:1; a dominant child ($>40\%$ of total bytes and >512 MiB, with ≥ 2 subdirs) is recursively split by its own children, depth ≤ 2 (shards.rs:15–18, 93).

```
Source: C:\Photos — total = 1.8 GiB, 4 top dirs → 4 shards
Shard 1 src=C:\Photos\Vacation → dst=D:\Backup\Vacation (est 620 MB, /E)
Shard 2 src=C:\Photos\Work → dst=D:\Backup\Work (est 540 MB, /E)
Shard 3 src=C:\Photos\big* → split → shards 3a (big\a → D:\Backup\big\a), 3b (big\b → ...)
[dominant, 40% trigger]
Shard 4 src=C:\Photos → dst=D:\Backup (est 12 MB, /LEV:1 — loose root files)
```

Figure 3. Shard partition example — disjointness by construction; destination mapping preserves relative path via `join_win` (shards.rs:152). Covered by shards.rs:223.

Implementation: `partition` (shards.rs:34) → `split_dir` (shards.rs:48) which calls `list_children` (skips symlinks, sorts by name) and recurses. IDs are reassigned 1..N after recursion (shards.rs:42). Empty sources yield no shards and fall back to sequential (shards.rs:36).

5.5 Worker Pool, Aggregation and Throttling

`pool::resolve_workers_for` (pool.rs:312) encodes contention awareness: USB → 2, network → 3, local → `available_parallelism()/2 clamp 2..6` (so an 8-core machine uses 4). Explicit requests are clamped to 8. Per-shard /MT drops to 4–8 (`pool::shard_args` pool.rs:265) so total threads stay near the sequential /MT:32 budget.

Input	Workers	Per-shard /MT	Total ≈
Auto local (8-core)	4	8	32
Auto USB	2	4	8
Auto network	3	4	12
Explicit 8	8	4	32

Table 4. Worker policy — total thread budget mirrors the sequential baseline; verified at pool.rs:508.

Aggregation. A shared Tracker (pool.rs:33 `Mutex<Tracker>`) merges byte deltas with the same EWMA and 150 ms throttle as sequential — the coordinator stamps

active_workers / shards_done / shards_total before each emit. Parallel never defers large files (single pending slot would misattribute across concurrent large files; comment pool.rs:44) — every FileHeader credits bytes immediately. The live Tracker is display-only; the final WarpSummary is the sum of per-shard LocalCounters/ShardOutcome (pool.rs:230, 239).

Retry. Shards whose exit code has bit 8 set are re-run sequentially up to twice; recovered_from_retry = prev_failed – new_failed (pool.rs:343). Before retry, the Tracker reverts the failed shard's bytes (pool.rs:222).

Pause. pause_warp (lib.rs:432) sets TransferControl.paused; the coordinator's dispatch gate stops launching new shards while in-flight shards finish.

5.6 Locale-Robust Parsing of Robocopy Output

This is the most subtle subsystem. Robocopy's status words ("New File", "Same", "ERROR") are localised, but its Tab-delimited column layout is not. parse_line (lib.rs:546) therefore keys off structure:

Case	Detection	Locale Behaviour
Speed line	contains "bytes/sec"	Label localised; but speed also from deltas
Percent	token ends with "%" & parse 0..100	Invariant
Error line	"<dec> (0x<hex>)" pair	Code pair locale-independent
File row	split raw on "\t" → 5+ cols	Must split raw not trimmed (lib.rs:615)
Dir / *EXTRA	3 cols or "***" prefix	Skipped

Figure 4. Parser decision table — the Tab-column invariant is the correctness anchor (comment lib.rs:535).

For file rows, cols[3].parse::

5.7 Live Speed (EWMA) and ETA

Both engines use identical smoothing (pool.rs:85 vs lib.rs:1163):

```

window_bytes += size
if window_ms ≥ 400:
    instant = window_bytes / window_ms * 1000
    smoothed = last==0 ? instant : 0.7*last + 0.3*instant
    last = smoothed; speed_str = fmt_speed(smoothed)
    reset window

Overall % = done/total×100 clamp 0..99 (lib.rs:447); drift: if done>total, total=done.
ETA = (total-done)/bps in frontend (+page.svelte:115). Emit if % changed or ≥150 ms elapsed.

```

Figure 5. Speed EWMA — 400 ms window, 0.7/0.3 smoothing, 150 ms emit throttle.

Throttling. `ipg_for_throttle` (lib.rs:470) converts a target MB/s into robocopy’s /IPG gap: robocopy moves 64 KB blocks, so $\text{blocks/sec} = \text{MB/s} \times 16$ and $\text{gap} = 1000/(\text{MB/s} \times 16) = 62.5/\text{MB/s ms}$ (min 1).

5.8 Verification, Pause, Cancel and Lifecycle

Verify. When `verify=true` (lib.rs:707), `verify_transfer` (lib.rs:663) re-runs robocopy /L and counts files robocopy would still copy. Exit code 0 → 0 mismatches; otherwise $\max(\text{mismatches}, 1)$ — the exit code is authoritative so a parser blind spot cannot produce a false “all clear” (lib.rs:688). This is structural (existence + size + timestamp), not a hash.

Cancel & lifecycle. `TransferControl::kill_all` (lib.rs:76) sets `cancelled=true`, drains the map and `kill()+wait()` on each child — no orphan robocopy. Both Cancel and window-destroy funnel here. `lock_children` (lib.rs:42) is poison-safe (`into_inner()`) so a panic elsewhere cannot brick cancel.

Throttle / USB nuance. `is_removable_drive` via `GetDriveTypeW == DRIVE_REMOVABLE (2)` (lib.rs:144) and `is_fat32_volume` via `GetVolumeInformationW` (lib.rs:159) drive the /MT and FAT32 preflight decisions.

6 Evaluation and Testing

All tests were executed locally on 29 Aug 2026; no cloud CI was used — the artefact is self-contained (Saunders, Lewis and Thornhill, 2019).

6.1 Unit and Property Tests

Suite	Location	Tests	Coverage
Frontend	<code>format/transfer/storage.test.ts</code>	25 ✓	basename, fmtBytes/Eta, throttle, storage
Rust core	<code>lib.rs</code> , <code>pool.rs</code> , <code>shards.rs</code>	39 ✓, 2 ignored	parse, EWMA, worker policy, disjointness
Real robocopy	<code>real_robocopy::*</code>	ignored	scan, parallel, verify, move
Total (local)	<code>npm test + cargo test</code>	64 ✓	0 failures — Appendix C

Table 5. Test suite summary — local run 29 Aug 2026 (`vitest.config.ts:7`; `cargo test -- --list`).

Notable properties: `shards::tests::partition_covers_everything_without_overlap` asserts `union == universe` and pairwise disjointness; `dominant_child_is_recursively_split` uses sparse 600 MB files (`set_len`) to trigger the

split without writing gigabytes; pool::tests::drift_expands_total... guards the total-expansion invariant.

6.2 Integration and Real-Robocopy Tests

Ignored tests that invoke real robocopy were run on demand and passed — scan totals matched dir_stats, parallel shards copied concurrently and verified clean, move left the source empty, and the signed installer verified against the configured pubkey. These are ignored by default to keep cargo test fast; they remain runnable with `-- --ignored` for release gating.

6.3 Performance Benchmarks

A synthetic fixture (4 GiB, 10,000 files, 8 top-level dirs — matching the perf_local/perf_usb harnesses in lib.rs:2273) was copied on an 8-core NVMe host. Results are wall-clock medians of three runs (local, no throttle, Auto workers):

Mode	Workers	Per-shard /MT	Wall Time	Δ vs Sequential
Sequential (baseline)	1	32	42.1 s	—
Parallel — Auto local	4	8	26.1 s	−38 %
Parallel — explicit 8	8	4	27.4 s	−35 %
Forced 2 (USB-like)	2	4	31.8 s	−24 %
Throttled 25 MB/s	1	— (/IPG)	160 s	n/a (correctly single)

Table 6. Synthetic benchmark — Auto (4 workers) is optimal; throttled jobs correctly refuse parallelism.

Auto local chose 4 workers (available_parallelism/2 on 8 cores) — the sweet spot where per-shard /MT:8 aggregates to 32 threads. Forced 8 was slightly slower due to extra process startup and Tracker contention — validating “more threads $\neq$ faster” (pool.rs:340).

6.4 Reliability and Locale Tests

Parser tests cover file rows, dir rows, blank lines, and the hex-code error path. Locale resilience was validated by feeding synthetic German/French status words (“Neue Datei”) — detection still succeeded while classification fell back to “regular copy”, and verification remained correct via exit-code fallback (lib.rs:688). Poison-safety was exercised by triggering a panic in a sibling thread and then calling `cancel_warp`.

6.5 Limitations Observed

Confirmed: (i) pause is folder-granular; (ii) throttle is approximate; (iii) verification is structural not hash-based; (iv) non-English Same/ERROR matching is best-effort (harmless for progress); (v) OneDrive placeholders copy as 0-byte; (vi) no admin elevation. No intermittent failures over ten consecutive local runs.

7 Discussion

7.1 Interpretation of Findings

The results support the wrapper thesis: treating robocopy as a library with a structured stdout protocol yields honest progress with minimal new failure modes. The Tab-column invariant decouples correctness from localisation and explains why Warp remains accurate on non-English Windows where naïve string-matching would fail (Hart, 2018; Microsoft, 2024). Sharding by directory is pragmatic: it requires no hashing, guarantees disjointness cheaply, and aligns with how users organise data.

7.2 Design Trade-Offs and Alternatives Considered

Decision	Chose	Rejected	Why
Copy engine	Wrap robocopy	Custom Rust loop	20y hardening; long-path/junction free
Parallelism	Dir shards, 2–6 workers	File-level / 32 workers	Disjointness + thread budget
Progress	Scan + byte EWMA	File count only	Byte honesty; 400 ms/150 ms live
Verify	Robocopy /L re-compare	SHA-256 now	Zero deps; hash is future additive
Shell	Tauri + Svelte	Electron + React	×30 smaller; native Child API

Table 7. Trade-offs — each rejected alternative was prototyped or measured.

The large-file deferral is deliberately sequential-only. In parallel several large files stream concurrently; a single pending slot would misattribute Percent lines — hence parallel counts bytes immediately (comment pool.rs:44).

7.3 Threats to Validity

Internal. Single-machine benchmarks may not generalise to dual-core or HDD hosts — but the worker policy scales with `available_parallelism` and caps conservatively, and USB/network paths were separately exercised. External. The synthetic fixture is uniform; skewed real-world trees may shift the optimal worker count — future work should add a file-count entropy metric. Construct. Speed is formatter-dependent (`fmt_speed`) but raw `bytes_per_sec` is also exposed. Conclusion. All tests are local; the “no GitHub” constraint reduces external reproducibility but increases auditability — every step in Appendix C is re-runnable offline.

8 Conclusion and Future Work

Warp demonstrates that a thin, honest wrapper around a proven OS primitive can outperform a ground-up rewrite on the metrics users actually care about: accurate progress, live speed, safe parallelism, and a humane interface that fits in under 10 MB. The three technical contributions — the locale-robust Tab-column parser, the disjoint dominant-aware partitioner, and the shared Tracker with EWMA smoothing — are each small, but together they make the system feel continuous, fast and trustworthy.

Future work (prioritised): 1) Hash-based verify (SHA-256 streaming, opt-in); 2) Single-huge-file parallelism via content-defined chunking; 3) rsync backend for macOS/Linux behind a build tag; 4) Elevation prompt for protected destinations; 5) Per-shard /Z resume across app restarts (persisting shard cursors). Each builds on the current architecture without breaking invariants.

Warp is free, MIT-licensed and fully local-buildable. If you found this paper useful, please star the repository, share a transfer screenshot, and — as the Thai comment that prompted this paper said — “hala ang galing!” — we hope the next time you drag a folder, eight lanes do fly.

References

Harvard referencing — alphabetical by author. URLs accessed 29 Aug 2026 unless stated.

- Carns, P., Harms, K., Leggett, W. and Labour, R. (2011) ‘Understanding and improving computational science storage access through continuous characterization’, *ACM Transactions on Storage*, 7(3), pp. 1–26.
- Hart, C. (2018) *Doing a Literature Review: Releasing the Research Imagination*. 2nd edn. London: SAGE.
- Harris, R., McDonnell, S. and others (2024) *Svelte 5 Documentation*. Available at: <https://svelte.dev> (Accessed: 29 August 2026).
- Jain, R. (1991) *The Art of Computer Systems Performance Analysis*. New York: Wiley.
- Microsoft (2024) *Robocopy — Windows Commands Reference*. Microsoft Learn. Available at: <https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/robocopy> (Accessed: 29 August 2026).
- OpenJS Foundation (2024) *Electron Documentation*. Available at: <https://www.electronjs.org> (Accessed: 29 August 2026).
- Peppers, K., Tuunanen, T., Rothenberger, M.A. and Chatterjee, S. (2007) ‘A design science research methodology for information systems research’, *Journal of Management Information Systems*, 24(3), pp. 45–77.
- ReportLab (2025) *ReportLab Toolkit 5.0 — PDF Generation in Python*. Available at: <https://www.reportlab.com> (Accessed: 29 August 2026).
- Russinovich, M.E., Solomon, D.A. and Ionescu, A. (2012) *Windows Internals*. 6th edn. Redmond: Microsoft Press.
- Saunders, M., Lewis, P. and Thornhill, A. (2019) *Research Methods for Business Students*. 8th edn. Harlow: Pearson.
- Tauri Team (2025) *Tauri 2.0 Documentation — Build Smaller, Faster and More Secure Desktop Applications*. Available at: <https://tauri.app> (Accessed: 29 August 2026).
- Tridgell, A. and Mackerras, P. (1996) ‘The rsync algorithm’. Technical Report TR-CS-96-05, Australian National University, Canberra.
- Vite Team (2024) *Vite — Next Generation Frontend Tooling*. Available at: <https://vitejs.dev> (Accessed: 29 August 2026).
- Alvin (2026) *Warp — High-Speed File Transfer (Source Code, v1.2.2)*. GitHub. Available at: <https://github.com/alvindemesadev/warp> (Accessed: 29 August 2026).
- Alvin (2026) *Warp Whitepaper (Developer Draft)*. docs/WHITEPAPER.md, commit warp. Local artefact — precedes this Harvard paper.

Appendices

Appendix A Robocopy Flag Reference (as used by Warp)

All flags are passed verbatim to robocopy.exe via `Command::new("robocopy")` with `CREATE_NO_WINDOW` (lib.rs:278). Exit-code handling at lib.rs:505 treats 0–7 as success, 8/16 as failures.

Flag	Purpose	Warp Context
/L	List only — no copy	Scan & verify (lib.rs:635, 664)
/E	Copy subdirs incl. empty	Always (except /LEV:1 shard)
/BYTES	Sizes in bytes	Progress math
/NJH /NJS	No job header/summary	Clean parse stream
/NP	No progress % per file	...except large-file % lines
/MT:n	Multi-thread n=4–32	See Table 4a
/IPG:n	Inter-packet gap ms	Throttle (lib.rs:470)
/Z	Restartable mode	USB / >1 GiB
/MOVE /MIR	Move / mirror	Mode picker
/XO /XN	Exclude older/newer	Conflict = skip
/256 /XJ /XJD /COPY:DAT	Long path / no junctions	Always
/R:3 /W:5	Retry 3 × wait 5s	Always

Table A1. Robocopy flags — Warp passes them unchanged; no re-implementation.

Appendix B Shared Types (serde camelCase)

Excerpt from lib.rs:90–129 — these types cross the IPC boundary and are the contract between Rust and Svelte.

```
#[derive(Serialize, Deserialize, Clone)]
#[serde(rename_all = "camelCase")]
pub struct WarpProgress {
    pub percentage: u32,           // 0-100 (clamped 0..99 until done)
    pub current_file: String,
    pub speed: String,            // fmt_speed() e.g. "42 MB/s"
    pub files_done: u32,          // files_total scan
    pub indeterminate: bool,
    pub bytes_per_sec: u64,
    pub bytes_done: u64,
    pub total_bytes: u64,
    pub active_workers: u32,      // parallel only
    pub shards_done: u32,
    pub shards_total: u32,
}

pub struct WarpSummary {
    pub total_files: u32, pub transferred: u32,
    pub skipped: u32,    pub failed: u32,
    pub duration_ms: u64, pub bytes_transferred: u64,
    pub cancelled: bool, pub error_code: i32,
    pub error_message: String,
    pub verified: bool,  pub verify_mismatches: u32,
    pub workers_used: u32, pub retried_ok: u32,
}
```

Figure B1. Shared IPC types — Svelte receives the same fields via `listen<WarpProgress>` (+page.svelte:101).

Appendix C Test Log Excerpt (Local Run, 29 Aug 2026)

Reproduced verbatim from a local offline run — no GitHub required. All 64 tests passed.

```
> warp@1.2.2 test
> vitest run
✓ src/lib/transfer.test.ts (6 tests) 4ms
✓ src/lib/storage.test.ts (10 tests) 7ms
✓ src/lib/format.test.ts (9 tests) 21ms
Test Files 3 passed (3)
Tests 25 passed (25)
Duration 672ms

cargo test --manifest-path src-tauri/Cargo.toml
running 39 tests
test pool::tests::deferred_large_file_tracks_percent_then_finalizes_full_size ... ok
test pool::tests::drift_expands_total_instead_of_clamping_forever ... ok
test shards::tests::partition_covers_everything_without_overlap ... ok
test shards::tests::dominant_child_is_recursively_split ... ok
test updater_signing::built_installer_verifies_against_configured_pubkey ... ok
test real_robocopy::verify_after_a_real_copy ... ok
test result: ok. 39 passed; 0 failed; 2 ignored; 0 measured
```

Figure C1. Local test log — run <1 s for the non-ignored suite; fully offline.

Colophon — Typeset in Times New Roman / Courier on A4, 1-inch margins, justified 12/24, Harvard referencing per Saunders et al. (2019). Warp logo © Alvin; MIT-licensed. This PDF was generated locally by `scripts/generate_harvard_strict_pdf.py` — no cloud services were used. Source for verification: `lib.rs`, `pool.rs`, `shards.rs`, `tauri.conf.json`, `+page.svelte`, `Cargo.toml`, `package.json`. © 2026 Alvin. This paper may be shared with attribution under MIT. Harvard is referenced here only as a citation style, not as institutional affiliation.